

Numerical Methods

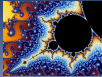
Lecture Notes #08

Discrete Least Square Approximation

Pavel Ludvík,
<pavel.ludvik@vsb.cz>

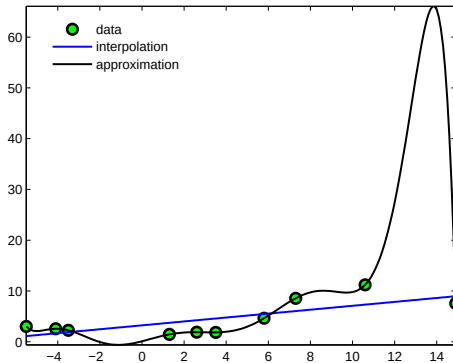
March 30, 2016
Department of Mathematics and Descriptive Geometry
VŠB-TUO

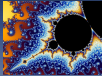
<http://homen.vsb.cz/lud0016/>



Introduction: Matching a Few Parameters to a Lot of Data.

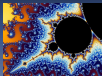
Sometimes we get a lot of data, **many observations**, and want to fit it to a simple model.





Why a Low Dimensional Model?

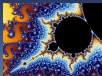
Low dimensional models (*e.g.* low degree polynomials) are **easy to work with**, and are quite **well behaved** (high degree polynomials can be quite oscillatory).



Why a Low Dimensional Model?

Low dimensional models (*e.g.* low degree polynomials) are **easy to work with**, and are quite **well behaved** (high degree polynomials can be quite oscillatory).

All measurements are **noisy**, to some degree. Often, we want to use a large number of measurements in order to "average out" random noise.



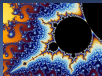
Why a Low Dimensional Model?

Low dimensional models (*e.g.* low degree polynomials) are **easy to work with**, and are quite **well behaved** (high degree polynomials can be quite oscillatory).

All measurements are **noisy**, to some degree. Often, we want to use a large number of measurements in order to "average out" random noise.

Approximation Theory looks at two problems:

- [1] Given a data set, find the best fit for a model (*i.e.* in a class of functions, find the one that best represents the data.)



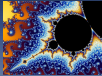
Why a Low Dimensional Model?

Low dimensional models (*e.g.* low degree polynomials) are **easy to work with**, and are quite **well behaved** (high degree polynomials can be quite oscillatory).

All measurements are **noisy**, to some degree. Often, we want to use a large number of measurements in order to "average out" random noise.

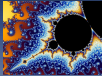
Approximation Theory looks at two problems:

- [1] Given a data set, find the best fit for a model (*i.e.* in a class of functions, find the one that best represents the data.)
- [2] Find a simpler model approximating a given function.



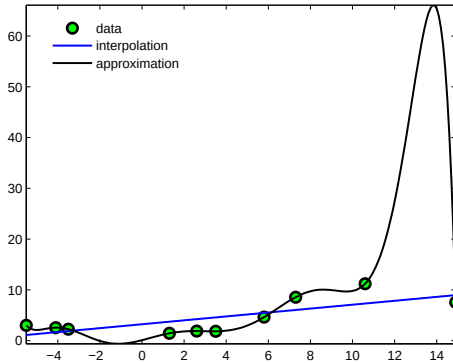
Interpolation: A Bad Idea?

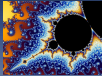
We can probably agree that trying to interpolate this data set:



Interpolation: A Bad Idea?

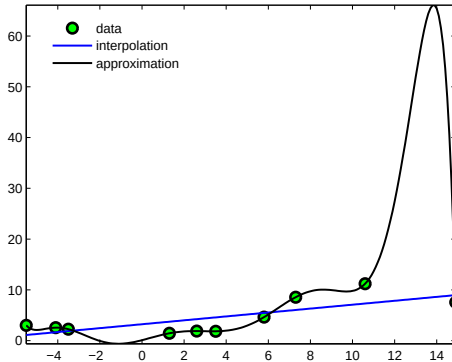
We can probably agree that trying to interpolate this data set:



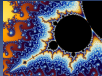


Interpolation: A Bad Idea?

We can probably agree that trying to interpolate this data set:



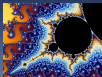
with a 9th degree polynomial is not the best idea in the world. . .
So what do we do?



Defining “Best Fit” – the Residual.

We are going to **relax** the requirement that the approximating function must pass through all the data points.

Now we need a measurement of how well our approximation fits the data. – A definition of “best fit”.



Defining “Best Fit” – the Residual.

We are going to **relax** the requirement that the approximating function must pass through all the data points.

Now we need a measurement of how well our approximation fits the data. – A definition of “best fit”.

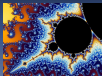
If $f(x_i)$ are the measured function values, and $a(x_i)$ are the values of our approximating functions, we can define a function, $r(x_i) = f(x_i) - a(x_i)$ which measures the deviation (**residual**) at x_i . Notice that

$\tilde{\mathbf{r}} = \{r(x_0), r(x_1), \dots, r(x_n)\}^T$ is a vector.

Notation:

From now on, $f_i = f(x_i)$, $a_i = a(x_i)$, and $r_i = r(x_i)$.

Further, $\tilde{\mathbf{f}} = \{f_0, f_1, \dots, f_n\}^T$, $\tilde{\mathbf{a}} = \{a_0, a_1, \dots, a_n\}^T$, and $\tilde{\mathbf{r}} = \{r_0, r_1, \dots, r_n\}^T$.



What is the Size of the Residual

There are many possible reasonable choices, *e.g.*

- The abs-sum of the deviations:

$$E_1 = \sum_{i=0}^n |r_i| \equiv E_1 = \|\tilde{\mathbf{r}}\|_1$$

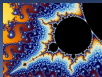
- The sum-of-squares of the deviations:

$$E_2 = \sqrt{\sum_{i=0}^n |r_i|^2} \equiv E_2 = \|\tilde{\mathbf{r}}\|_2$$

- The largest of the deviations:

$$E_\infty = \max_{0 \leq i \leq n} |r_i| \equiv E_\infty = \|\tilde{\mathbf{r}}\|_\infty$$

In most cases, the **sum-of-the-squares** version is the easiest to work with. (From now on we will focus on this choice. . .)



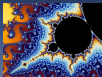
Discrete Least Squares Approximation

We have chosen the sum-of-squares measurement for errors. Lets find the **constant** that best fits the data, *minimize*

$$E(C) = \sum_{i=0}^n (f_i - C)^2.$$

If C^* is a minimizer, then $E'(C^*) = 0$ (since **derivative at a max/min is zero**).

$$E'(C) = - \sum_{i=0}^n 2(f_i - C) = -2 \sum_{i=0}^n f_i + 2(n+1)C, \quad E''(C) = 2(n+1)$$



Discrete Least Squares Approximation

We have chosen the sum-of-squares measurement for errors. Lets find the **constant** that best fits the data, *minimize*

$$E(C) = \sum_{i=0}^n (f_i - C)^2.$$

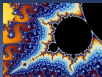
If C^* is a minimizer, then $E'(C^*) = 0$ (since **derivative at a max/min is zero**).

$$E'(C) = - \sum_{i=0}^n 2(f_i - C) = -2 \sum_{i=0}^n f_i + 2(n+1)C, \quad E''(C) = 2(n+1)$$

hence

$$C^* = \frac{1}{n+1} \sum_{i=0}^n f_i, \text{ it is a min since } E''(C^*) = 2(n+1) > 0,$$

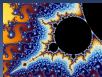
is the constant that best fits the data. (**Note:** C^* is the average.)



Discrete Least Squares: Linear Approximation

The form of Least Squares you are most likely to see: **Find the Linear Function**, $p_1(x) = a_0 + a_1x$, **that best fits the data**. The error $E(a_0, a_1)$ we need to minimize is:

$$E(a_0, a_1) = \sum_{i=0}^n [(a_0 + a_1 x_i) - f_i]^2.$$



Discrete Least Squares: Linear Approximation

The form of Least Squares you are most likely to see: **Find the Linear Function**, $p_1(x) = a_0 + a_1x$, **that best fits the data**. The error $E(a_0, a_1)$ we need to minimize is:

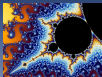
$$E(a_0, a_1) = \sum_{i=0}^n [(a_0 + a_1 x_i) - f_i]^2.$$

The first partial derivatives with respect to a_0 and a_1 better be zero at the minimum:

$$\frac{\partial}{\partial a_0} E(a_0, a_1) = 2 \sum_{i=0}^n [(a_0 + a_1 x_i) - f_i] = 0$$

$$\frac{\partial}{\partial a_1} E(a_0, a_1) = 2 \sum_{i=0}^n x_i [(a_0 + a_1 x_i) - f_i] = 0.$$

Now, we work on these expressions to get the **Normal Equations**...

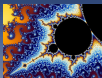


Linear Approximation: The Normal Equations

$p_1(x)$

$$\sum_{i=0}^n [(a_0 + a_1 x_i) - f_i] = 0$$

$$\sum_{i=0}^n x_i [(a_0 + a_1 x_i) - f_i] = 0.$$

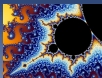


Linear Approximation: The Normal Equations

$p_1(x)$

$$\begin{aligned} a_0(n+1) + a_1 \sum_{i=0}^n x_i &= \sum_{i=0}^n f_i \\ a_0 \sum_{i=0}^n x_i + a_1 \sum_{i=0}^n x_i^2 &= \sum_{i=0}^n x_i f_i \end{aligned}$$

Since **everything except** a_0 and a_1 is known, this is a 2-by-2 system of equations.



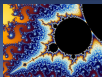
Linear Approximation: The Normal Equations

$p_1(x)$

$$\begin{aligned} a_0(n+1) + a_1 \sum_{i=0}^n x_i &= \sum_{i=0}^n f_i \\ a_0 \sum_{i=0}^n x_i + a_1 \sum_{i=0}^n x_i^2 &= \sum_{i=0}^n x_i f_i \end{aligned}$$

Since **everything except** a_0 and a_1 is known, this is a 2-by-2 system of equations.

$$\begin{bmatrix} (n+1) & \sum_{i=0}^n x_i \\ \sum_{i=0}^n x_i & \sum_{i=0}^n x_i^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} \sum_{i=0}^n f_i \\ \sum_{i=0}^n x_i f_i \end{bmatrix}$$

**Quadratic Model, $p_2(x)$**

For the quadratic polynomial $p_2(x) = a_0 + a_1x + a_2x^2$, the error is given by

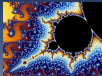
$$E(a_0, a_1, a_2) = \sum_{i=0}^n [a_0 + a_1x_i + a_2x_i^2 - f_i]^2$$

At the minimum (best model) we must have

$$\frac{\partial}{\partial a_0} E(a_0, a_1, a_2) = 2 \sum_{i=0}^n [(a_0 + a_1x_i + a_2x_i^2) - f_i] = 0$$

$$\frac{\partial}{\partial a_1} E(a_0, a_1, a_2) = 2 \sum_{i=0}^n x_i [(a_0 + a_1x_i + a_2x_i^2) - f_i] = 0$$

$$\frac{\partial}{\partial a_2} E(a_0, a_1, a_2) = 2 \sum_{i=0}^n x_i^2 [(a_0 + a_1x_i + a_2x_i^2) - f_i] = 0$$



Quadratic Model: The Normal Equations

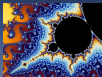
 $p_2(x)$

Similarly for the quadratic polynomial $p_2(x) = a_0 + a_1x + a_2x^2$, the normal equations are:

$$\begin{aligned}a_0(n+1) + a_1 \sum_{i=0}^n x_i + a_2 \sum_{i=0}^n x_i^2 &= \sum_{i=0}^n f_i \\a_0 \sum_{i=0}^n x_i + a_1 \sum_{i=0}^n x_i^2 + a_2 \sum_{i=0}^n x_i^3 &= \sum_{i=0}^n x_i f_i \\a_0 \sum_{i=0}^n x_i^2 + a_1 \sum_{i=0}^n x_i^3 + a_2 \sum_{i=0}^n x_i^4 &= \sum_{i=0}^n x_i^2 f_i\end{aligned}$$

Note:

Even though the model is quadratic, the resulting (normal) equations are **linear** – The model is linear in its parameters, a_0 , a_1 and a_2 .



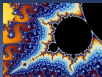
The Normal Equations – As Matrix Equations

We have:

$$a_0(n+1) + a_1 \sum_{i=0}^n x_i + a_2 \sum_{i=0}^n x_i^2 = \sum_{i=0}^n f_i$$

$$a_0 \sum_{i=0}^n x_i + a_1 \sum_{i=0}^n x_i^2 + a_2 \sum_{i=0}^n x_i^3 = \sum_{i=0}^n x_i f_i$$

$$a_0 \sum_{i=0}^n x_i^2 + a_1 \sum_{i=0}^n x_i^3 + a_2 \sum_{i=0}^n x_i^4 = \sum_{i=0}^n x_i^2 f_i$$

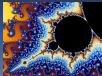


The Normal Equations – As Matrix Equations

We rewrite the Normal Equations as:

$$\begin{bmatrix} (n+1) & \sum_{i=0}^n x_i & \sum_{i=0}^n x_i^2 \\ \sum_{i=0}^n x_i & \sum_{i=0}^n x_i^2 & \sum_{i=0}^n x_i^3 \\ \sum_{i=0}^n x_i^2 & \sum_{i=0}^n x_i^3 & \sum_{i=0}^n x_i^4 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \sum_{i=0}^n f_i \\ \sum_{i=0}^n x_i f_i \\ \sum_{i=0}^n x_i^2 f_i \end{bmatrix}$$

It is not immediately obvious, but this expression can be written in the form $A^T A \tilde{a} = A^T \tilde{f}$. Where matrix A is very easy to write in terms of x_i .



The Polynomial Equations in Matrix Form

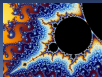
$$p_m(x)$$

We can express the m_{th} degree polynomial, $p_m(x)$, evaluated at the points x_i :

$$a_0 + a_1x_i + a_2x_i^2 + \cdots + a_mx_i^m = f_i, \quad i = 0, \dots, n$$

as a product of an $(n+1)$ -by- $(m+1)$ matrix, A and the $(m+1)$ -by-1 vector $\tilde{\mathbf{a}}$ and the result is the $(n+1)$ -by-1 vector $\tilde{\mathbf{f}}$, usually $n \gg m$:

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^m \\ 1 & x_1 & x_1^2 & \cdots & x_1^m \\ 1 & x_2 & x_2^2 & \cdots & x_2^m \\ 1 & x_3 & x_3^2 & \cdots & x_3^m \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^m \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_m \end{bmatrix}$$



Building a Solvable System from $A\tilde{\mathbf{a}} = \tilde{\mathbf{f}}$

We cannot immediately solve the linear system

$$A\tilde{\mathbf{a}} = \tilde{\mathbf{f}}$$

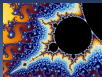
when A is a rectangular matrix $(n+1)$ -by- $(m+1)$, $m \neq n$.

We can generate a solvable system by multiplying both the left- and right-hand-side by A^T , *i.e.*

$$A^T A \tilde{\mathbf{a}} = A^T \tilde{\mathbf{f}}$$

Now, the matrix $A^T A$ is a square $(m+1)$ -by- $(m+1)$ matrix, and $A^T \tilde{\mathbf{f}}$ an $(m+1)$ -by-1 vector.

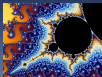
Let's take a closer look at $A^T A$, and $A^T \tilde{\mathbf{f}} \dots$



Computing $A^T A$

$$\begin{bmatrix}
 \color{red}{1} & \color{red}{1} & \color{red}{1} & \color{red}{1} & \cdots & \color{red}{1} \\
 x_0 & x_1 & x_2 & x_3 & \cdots & x_n \\
 x_0^2 & x_1^2 & x_2^2 & x_3^2 & \cdots & x_n^2 \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 x_0^m & x_1^m & x_2^m & x_3^m & \cdots & x_n^m
 \end{bmatrix}
 \begin{bmatrix}
 \color{red}{1} & x_0 & x_0^2 & \cdots & x_0^m \\
 \color{red}{1} & x_1 & x_1^2 & \cdots & x_1^m \\
 \color{red}{1} & x_2 & x_2^2 & \cdots & x_2^m \\
 \color{red}{1} & x_3 & x_3^2 & \cdots & x_3^m \\
 \vdots & \vdots & \vdots & \vdots & \vdots \\
 \color{red}{1} & x_n & x_n^2 & \cdots & x_n^m
 \end{bmatrix}$$

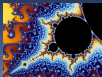
$$= \begin{bmatrix}
 \color{red}{n+1} & \circ & \circ & \circ \\
 \circ & \circ & \circ & \circ \\
 \circ & \circ & \circ & \circ \\
 \circ & \circ & \circ & \circ
 \end{bmatrix}$$



Computing $A^T A$

$$\begin{bmatrix}
 1 & 1 & 1 & 1 & \cdots & 1 \\
 x_0 & x_1 & x_2 & x_3 & \cdots & x_n \\
 x_0^2 & x_1^2 & x_2^2 & x_3^2 & \cdots & x_n^2 \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 x_0^m & x_1^m & x_2^m & x_3^m & \cdots & x_n^m
 \end{bmatrix}
 \begin{bmatrix}
 1 & x_0 & x_0^2 & \cdots & x_0^m \\
 1 & x_1 & x_1^2 & \cdots & x_1^m \\
 1 & x_2 & x_2^2 & \cdots & x_2^m \\
 1 & x_3 & x_3^2 & \cdots & x_3^m \\
 \vdots & \vdots & \vdots & \vdots & \vdots \\
 1 & x_n & x_n^2 & \cdots & x_n^m
 \end{bmatrix}$$

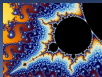
$$= \begin{bmatrix}
 n+1 & \circ & \circ & \sum_{i=0}^n x_i^m \\
 \circ & \circ & \circ & \circ \\
 \circ & \circ & \circ & \circ \\
 \circ & \circ & \circ & \circ
 \end{bmatrix}$$



Computing $A^T A$

$$\begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ x_0 & x_1 & x_2 & x_3 & \cdots & x_n \\ x_0^2 & x_1^2 & x_2^2 & x_3^2 & \cdots & x_n^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_0^m & x_1^m & x_2^m & x_3^m & \cdots & x_n^m \end{bmatrix} \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^m \\ 1 & x_1 & x_1^2 & \cdots & x_1^m \\ 1 & x_2 & x_2^2 & \cdots & x_2^m \\ 1 & x_3 & x_3^2 & \cdots & x_3^m \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^m \end{bmatrix}$$

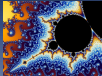
$$= \begin{bmatrix} n+1 & \circ & \circ & \sum_{i=0}^n x_i^m \\ \circ & \circ & \circ & \circ \\ \circ & \circ & \circ & \circ \\ \sum_{i=0}^n x_i^m & \circ & \circ & \circ \end{bmatrix}$$



Computing $A^T A$

$$\begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ x_0 & x_1 & x_2 & x_3 & \cdots & x_n \\ x_0^2 & x_1^2 & x_2^2 & x_3^2 & \cdots & x_n^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_0^m & x_1^m & x_2^m & x_3^m & \cdots & x_n^m \end{bmatrix} \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^m \\ 1 & x_1 & x_1^2 & \cdots & x_1^m \\ 1 & x_2 & x_2^2 & \cdots & x_2^m \\ 1 & x_3 & x_3^2 & \cdots & x_3^m \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^m \end{bmatrix}$$

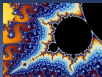
$$= \begin{bmatrix} n+1 & \circ & \circ & \sum_{i=0}^n x_i^m \\ \circ & \circ & \circ & \circ \\ \circ & \circ & \circ & \circ \\ \sum_{i=0}^n x_i^m & \circ & \circ & \sum_{i=0}^n x_i^{2m} \end{bmatrix}$$



Computing $A^T A$

$$\begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ x_0 & x_1 & x_2 & x_3 & \cdots & x_n \\ x_0^2 & x_1^2 & x_2^2 & x_3^2 & \cdots & x_n^2 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ x_0^m & x_1^m & x_2^m & x_3^m & \cdots & x_n^m \end{bmatrix} \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^m \\ 1 & x_1 & x_1^2 & \cdots & x_1^m \\ 1 & x_2 & x_2^2 & \cdots & x_2^m \\ 1 & x_3 & x_3^2 & \cdots & x_3^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^m \end{bmatrix}$$

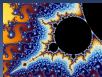
$$= \begin{bmatrix} n+1 & \sum_{i=0}^n x_i^1 & \circ & \sum_{i=0}^n x_i^m \\ \sum_{i=0}^n x_i^1 & \sum_{i=0}^n x_i^2 & \circ & \sum_{i=0}^n x_i^{m+1} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=0}^n x_i^m & \sum_{i=0}^n x_i^{m+1} & \circ & \sum_{i=0}^n x_i^{2m} \end{bmatrix}$$



Computing $A^T f$

$$\begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ x_0 & x_1 & x_2 & x_3 & \cdots & x_n \\ x_0^2 & x_1^2 & x_2^2 & x_3^2 & \cdots & x_n^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_0^m & x_1^m & x_2^m & x_3^m & \cdots & x_n^m \end{bmatrix} \begin{bmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_n \end{bmatrix} = \begin{bmatrix} \sum_{i=0}^n f_i \\ \sum_{i=0}^n x_i f_i \\ \sum_{i=0}^n x_i^2 f_i \\ \vdots \\ \sum_{i=0}^n x_i^m f_i \end{bmatrix}$$

Thus, we have recovered the Normal Equations...



Discrete Least Squares: A Simple, Powerful Method

Given the data set $(\tilde{\mathbf{x}}, \tilde{\mathbf{f}})$, where $\tilde{\mathbf{x}} = \{x_0, x_1, \dots, x_n\}^T$ and $\tilde{\mathbf{f}} = \{f_0, f_1, \dots, f_n\}^T$, we can quickly find the best polynomial fit for **any** specified polynomial degree!

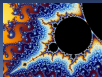
Notation: Let $\tilde{\mathbf{x}}^j$ be the vector $\{x_0^j, x_1^j, \dots, x_n^j\}^T$.

Example

To compute the best fitting polynomial of degree 3,
 $p_3(x) = a_0 + a_1x + a_2x^2 + a_3x^3$, define:

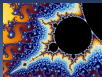
$$A = \begin{bmatrix} \vdots & \vdots & \vdots & \vdots \\ \tilde{\mathbf{1}} & \tilde{\mathbf{x}} & \tilde{\mathbf{x}}^2 & \tilde{\mathbf{x}}^3 \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}, \text{ and compute}$$

$$\tilde{\mathbf{a}} = (A^T A)^{-1} (A^T \tilde{\mathbf{f}}) \text{ (not necessarily like this.)}$$



But. . . I do not want to fit a polynomial!!!

Fitting an **exponential model** $g(x) = be^{cx}$ to the given data $\tilde{\mathbf{d}}$, is quite straight-forward.



But... I do not want to fit a polynomial!!!

Fitting an **exponential model** $g(x) = be^{cx}$ to the given data $\tilde{\mathbf{d}}$, is quite straight-forward.

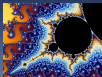
First, re-cast the problem as a set of linear equations. We have:

$$be^{cx_i} = d_i, \quad i = 0, \dots, n$$

compute the natural logarithm on both sides:

$$\ln b + cx_i = \ln d_i$$

$$a_0 + a_1 x_i = f_i$$



But... I do not want to fit a polynomial!!!

Fitting an **exponential model** $g(x) = be^{cx}$ to the given data $\tilde{\mathbf{d}}$, is quite straight-forward.

First, re-cast the problem as a set of linear equations. We have:

$$be^{cx_i} = d_i, \quad i = 0, \dots, n$$

compute the natural logarithm on both sides:

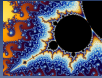
$$\ln b + cx_i = \ln d_i$$

$$a_0 + a_1 x_i = f_i$$

Now, we can apply a polynomial least squares fit to the problem and once we have (a_0, a_1) , $b = e^{a_0}$ and $c = a_1$.

Note

This does **not** give the least squares fit to the original problem!!! (It gives us a pretty good estimate.)



But... That is not a True Least Squares Fit!

Note

This does **not** give the least squares fit to the original problem!!! (It gives us a pretty good estimate.)

In order to find the true least squares fit we need to know how to find roots and/or minima/maxima of non-linear systems of equations. Feel free to look at Burden-Faires Chapter 10.